Google Developer Student Clubs

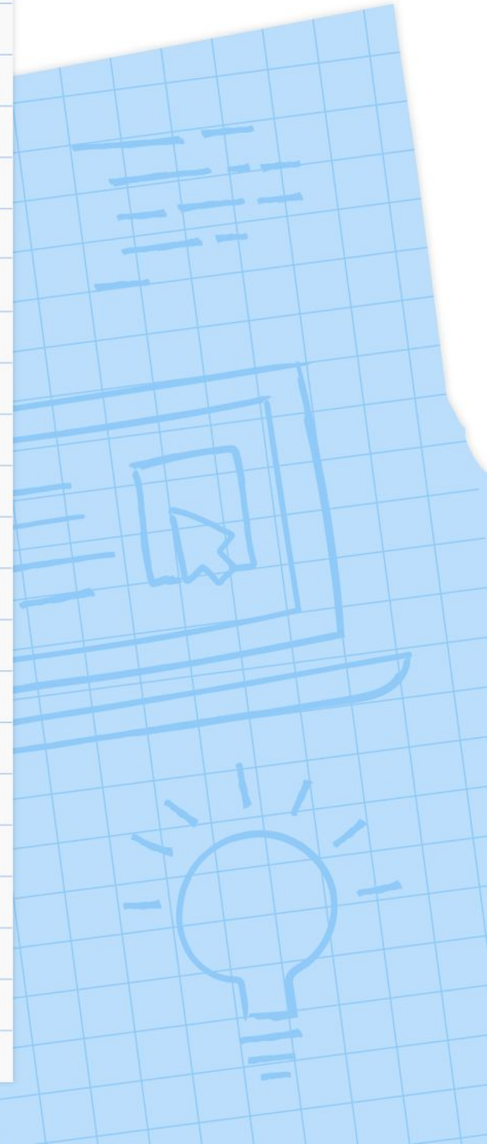
Szybkie budowanie aplikacji z Firebase i AI



Krzysztof Kaczyński

Agenda

- Firebase. What is this?
- What challenges Firebase solves?
- SQL vs Firebase
- Testing changes
- Security rules
- Unleash Google AI
- Live demo



Firebase. What is this?

- Traditional vs. Firebase



The Traditional Way

Deploying application infrastructure often demands setting up database management servers, API configurations, Docker, CORS policies, authentication routines, and secure server certificates from scratch.

Result: Hours or days lost just attempting to get your basic backend to talk to your client framework.

The Firebase Way

Plug in a single configuration file directly into your front-end. Immediate access to user login pathways, dynamic NoSQL data engines, and real-time streams with zero ops overhead.

Result: Fully integrated and secure authentication, hosting, and datastores within minutes of setup.

What challenges Firebase solves?

- The Backend Bottleneck

When is Firebase Best?

Rapid Prototyping & MVPs: Launch functional applications within standard 24-hour hackathons by eliminating complex server configurations.

Real-time Collaborations: Perfect for chats, document editors, gaming lobbies, and shared dashboards out of the box.

Offline-First Experiences: Firestore cache handles unexpected network drops transparently, writing to local memory and syncing later.

Front-end First Teams: Empowers client developers to execute fast changes without relying on backend teams or pipeline deploys.

When NOT to use Firebase?

Highly Relational Operations: If your core business requires strict ledger systems with deep nested queries, or multi-join relational computations, standard normalized relational databases are superior.
Alternative: Use PostgreSQL alongside Firebase for microservices.

Complex Processing: Serverless computing runs short, lightweight stateless logic. If you are doing continuous video transcoding or running prolonged background data pipelines, hosting functions gets expensive fast.

Alternative: Use dedicated Google Cloud VMs or GKE nodes.

No SQL vs SQL

- Relational vs Document

NoSQL vs. SQL Databases

Requirement	Cloud Firestore (NoSQL)	Traditional SQL Database
Schema Structure	Flexible documents & collections. Modify models on the fly.	Rigid tables, columns, constraints. Needs migrations.
Real-time Sync	Native WebSocket listening subscriptions built directly in.	Requires custom WebSockets, polling, or external brokers.
Scaling Vector	Automatic horizontal scale. Shards dynamically with loads.	Mostly vertical scaling (larger servers) unless clustered.
Complex Joins	Denormalize or write client-side query routines.	Exceptional at multi-table JOINs and deep analytics.

Unleash Google AI

- Genkit & Gemini Integration

Genkit & Gemini Integration

Combine Firebase with Google AI using Genkit. Build intelligent chatbots, structured LLM summary models, and automated context lookups seamlessly.

Vertex AI in Firebase lets you invoke Gemini models securely from client applications without exposing API keys.

Testing changes

- Local Emulator Suite

Local Emulator Suite

Local Host Routing: Run full replicas of Auth, Firestore, Cloud Storage, and Security Rules directly on local sandbox ports.

Emulator UI Dashboard: Visualize live local data mutations and trace user sessions using an elegant, locally hosted administrative dashboard.

Completely Offline Capabilities: Iterate code on trains or airplanes with zero dependence on active internet links.

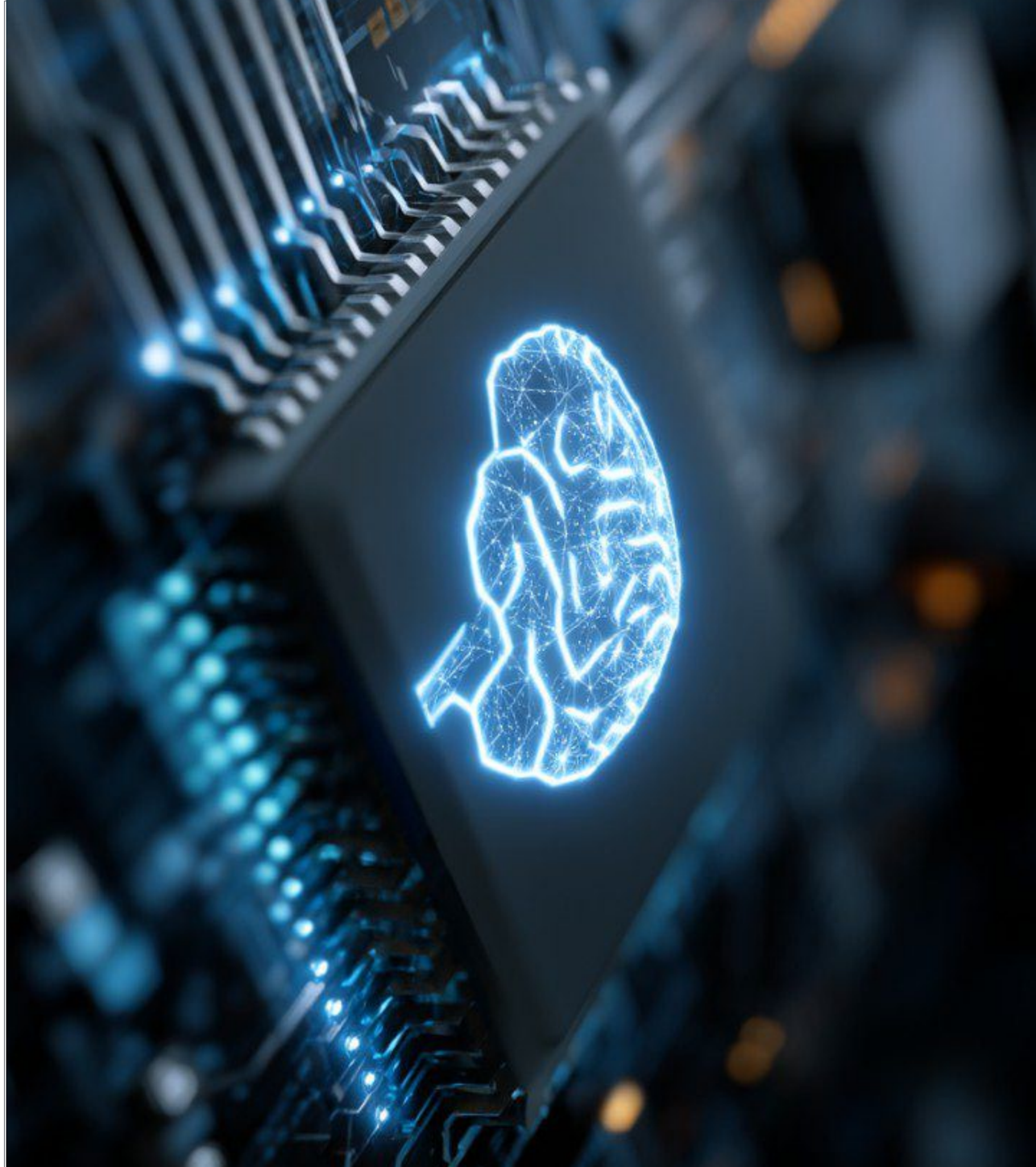
Test Security Locally: Verify firestore. rules logic against automated unit test suites before production push.

Live Demo

`Let's Build Faster!`

Grab the boilerplate and start shipping your next project in hours.

- Boilerplate: github.com/kaczor6418/fast-thon



Google Developer Student Clubs

THANK YOU
FOR YOUR TIME